Scratch3 射击游戏设计方案

文档目标

这是一份给 9 岁、三年级小朋友使用的 Scratch3 射击游戏学习方案。目标不是一下子做出一个超级复杂的大游戏，而是像真正的游戏开发者一样，按版本一步一步升级。
你会做出一款游戏，暂时给它起名叫：《太空小战士》
玩法很简单：

- 玩家控制一架小飞船左右移动
- 按空格键发射子弹
- 子弹打中敌人会得分
- 敌人碰到玩家会扣生命
- 后面版本再加入音效、关卡、Boss 和胜利画面
  这样设计有 3 个好处：
- 每个版本都能“马上运行起来”，不会做半天还玩不了
- 每个版本只学少量积木，容易理解
- 做完一个版本就有成就感，下一版再继续加功能

一、先认识这个 Scratch 项目工程

在开始做之前，先告诉你这个项目里会有哪些角色和变量。

1. 角色（Sprite）规划

建议你在项目里准备这些角色：

- `玩家飞船`
- `子弹`
- `敌人1`
- `开始按钮` （后面版本可加）
- `爆炸效果` （后面版本可加）
- `Boss` （最后版本可加）

2. 背景（Backdrop）规划

建议你准备这些背景：

- `首页`
- `太空战场`
- `游戏结束`

- `胜利`

3. 变量规划

点击左侧积木区的 **变量**，新建这些变量：

- `分数`
- `生命`
- `游戏状态`
- `敌人速度`
- `关卡`
  说明：
- `分数`： 记录打中了多少敌人
- `生命`： 玩家还能被撞几次
- `游戏状态`： 比如“开始中”“游戏中”“结束”
- `敌人速度`： 控制难度
- `关卡`： 记录第几关

4. 建议的工程命名

每做完一个版本，就另存一个文件，像这样：

- `space-v1.sb3`
- `space-v2.sb3`
- `space-v3.sb3`
  这样做的好处是：如果新版本做坏了，还能回到旧版本。

二、开发规则：每次只做一个能运行的小目标

我们把整个游戏拆成 8 个版本：

1. `V1` 玩家飞船能左右移动
2. `V2` 玩家可以发射子弹
3. `V3` 敌人出现并会移动
4. `V4` 子弹打中敌人并得分
5. `V5` 敌人撞到玩家会掉生命，能游戏结束
6. `V6` 增加开始界面和重新开始
7. `V7` 增加关卡、速度升级和音效
8. `V8` 增加 Boss 和胜利界面
   每一版都遵守这个顺序：
9. 先搭角色

10. 再写代码
11. 然后测试
12. 最后保存新版本

三、V1：先让玩家飞船动起来

本版本目标

玩家看到一艘飞船，按左右方向键就可以移动。

第 1 步：创建项目

打开 Scratch3 后：

1. 点击“创建”
2. 删除默认的小猫角色
3. 点击右下角“选择一个角色”
4. 找一个飞船，或者自己画一个简单飞船
5. 把角色名字改成 `玩家飞船`
6. 点击右下角“选择背景”
7. 选一个太空背景，命名为 `太空战场`

第 2 步：调整飞船位置和大小

先点中 `玩家飞船` 角色：

- 把大小改成 `50` 左右
- 把位置放到舞台下方
- 例如：`x=0`，`y=-140`

第 3 步：给玩家飞船写“开局初始化”代码

要用到的 Scratch 积木：

- **事件** 里的 `当绿旗被点击`
- **运动** 里的 `移到 x: y:`
- **外观** 里的 `显示`
  代码这样搭：

```
当绿旗被点击
显示
移到 x: 0 y: -140
```

意思是：每次点击绿旗，飞船都会回到起点。

第 4 步：让飞船按左右键移动

要用到的积木：

- **事件** 里的 `当绿旗被点击`
- **控制** 里的 `重复执行`
- **侦测** 里的 `按下 [右箭头] 键?`
- **侦测** 里的 `按下 [左箭头] 键?`
- **控制** 里的 `如果 那么`
- **运动** 里的 `将 x 坐标增加`
  代码这样搭：

```
当绿旗被点击
重复执行
如果 <按下 [右箭头] 键? > 那么
将 x 坐标增加 8
结束
如果 <按下 [左箭头] 键? > 那么
将 x 坐标增加 -8
结束
```

说明：

- 按右键时，`x` 变大，飞船往右走
- 按左键时，`x` 变小，飞船往左走
- `8` 是速度，可以改成 `6` 或 `10`

第 5 步：不让飞船跑出舞台

Scratch 舞台左右大约是：

- 最左边：`x=-240`
- 最右边：`x=240`
  所以要加边界判断。
  需要的积木：
- **运算** 里的 `< >`
- **控制** 里的 `如果 那么`
- **运动** 里的 `将 x 设为`
  在上面的循环里继续加入：

```
如果 <(x 坐标) > 220> 那么
将 x 设为 220
结束
如果 <(x 坐标) < -220> 那么
将 x 设为 -220
结束
```

第 6 步：测试 V1

测试方法：

1. 点击绿旗
2. 按左箭头
3. 按右箭头
4. 看飞船会不会从底部左右移动
5. 看它会不会冲出屏幕
   如果飞船太快：

- 把 `8` 改成 `5`
  如果飞船太慢：
- 把 `8` 改成 `10`

V1 完成标准

如果你已经做到下面 3 件事，就说明 V1 成功了：

- 点绿旗后飞船会回到起点
- 左右键能控制飞船
- 飞船不会跑出屏幕
  保存文件：
  `space-v1.sb3`

四、V2：让飞船发射子弹

本版本目标

按下空格键时，飞船可以发射子弹向上飞。

第 1 步：创建子弹角色

点击“选择一个角色”：

- 可以选择一个小圆点

- 或者自己画一个细长的激光
- 把角色名改成 `子弹`
  建议：
- 大小设置成 `20` 左右
- 方向朝上

第 2 步：先让子弹开局隐藏

选择 `子弹` 角色，加入代码：
要用的积木：

- **事件** `当绿旗被点击`
- **外观** `隐藏`

```
当绿旗被点击
隐藏
```

第 3 步：学会“克隆体”发射子弹

为什么不用一个子弹角色直接飞？
因为如果你只用一个子弹，上一颗还没飞走，就不能马上发第二颗。
Scratch 里解决方法是：
克隆自己
这样每按一次空格，就会产生一颗新的子弹。

第 4 步：按空格时创建子弹

仍然在 `子弹` 角色里写代码。需要的积木：

- **事件** `当绿旗被点击`
- **控制** `重复执行`
- **侦测** `按下 [空格] 键?`
- **控制** `如果 那么`
- **控制** `克隆自己`
- **控制** `等待 0.2 秒`

```
当绿旗被点击
重复执行
如果 <按下 [空格] 键? > 那么
克隆自己
```

```
等待 0.2 秒
结束
```

这里的 `等待 0.2 秒` 很重要。
作用是：

- 防止你按住空格时，一瞬间发出几百颗子弹

第 5 步：让克隆出来的子弹，从飞船位置发射

现在要让每颗子弹一出生，就出现在飞船头顶。
需要的积木：

- **控制** `当作为克隆体启动时`
- **运动** `移到 [玩家飞船]`
- **运动** `将 y 坐标增加`
- **外观** `显示`
- **控制** `重复执行`
- **运动** `将 y 坐标增加`
- **控制** `如果 那么`
- **运算** `>`
- **控制** `删除此克隆体`
  代码这样写：

```
当作为克隆体启动时
移到 [玩家飞船]
将 y 坐标增加 20
显示
重复执行
将 y 坐标增加 12
如果 <(y 坐标) > 180> 那么
删除此克隆体
结束
```

解释：

- `移到 [玩家飞船]`：找到飞船的位置
- `将 y 坐标增加 20`：让子弹出现在飞船头顶，不要和飞船重叠
- `将 y 坐标增加 12`：子弹一直往上飞
- 飞出屏幕后删除，节省资源

第 6 步：测试 V2

测试内容：

1. 点击绿旗
2. 按左右键移动飞船
3. 按空格发射
4. 看子弹是不是从飞船位置向上飞
5. 看子弹飞出舞台后会不会消失

V2 完成标准

- 飞船还能正常移动
- 按空格能不断发射子弹
- 子弹从飞船头部发出
- 子弹飞出上边界后消失
  保存文件：
  `space-v2.sb3`

五、V3：让敌人出现并移动

本版本目标

屏幕上方会不断出现敌人，敌人从上往下飞。

第 1 步：创建敌人角色

点击“选择一个角色”：

- 找一个外星飞船
- 或者自己画一个小怪物
- 命名为 `敌人1`
  建议大小：
- `40` 到 `55`

第 2 步：创建变量

点击 **变量**，新建：

- `分数`
- `生命`
- `游戏状态`
- `敌人速度`
  一开始先不用全部功能，但先建好，后面方便升级。

第 3 步：初始化变量

可以把初始化代码写在 `舞台背景`，也可以写在 `玩家飞船` 。推荐写在 `舞台背景` 。
选择舞台，加入：
需要的积木：

- **事件** `当绿旗被点击`
- **变量** `将 [分数] 设为`
- **变量** `将 [生命] 设为`
- **变量** `将 [游戏状态] 设为`
- **变量** `将 [敌人速度] 设为`

```
当绿旗被点击
将 [分数 v] 设为 0
将 [生命 v] 设为 3
将 [游戏状态 v] 设为 [游戏中]
将 [敌人速度 v] 设为 3
```

第 4 步：让敌人角色开局隐藏

在 `敌人1` 角色中：

```
当绿旗被点击
隐藏
```

第 5 步：持续生成敌人

在 `敌人1` 角色中加入：
需要积木：

- **事件** `当绿旗被点击`
- **控制** `重复执行`
- **控制** `克隆自己`
- **控制** `等待 1 秒`

```
当绿旗被点击
重复执行
如果 <(游戏状态) = [游戏中]> 那么
克隆自己
等待 1 秒
结束
```

这里还需要一个比较积木：

- **运算** `=`

第 6 步：让敌人从上方随机出现

在 `敌人1` 角色中加入克隆体代码：
需要积木：

- **控制** `当作为克隆体启动时`
- **运动** `移到 x: y:`
- **运算** `在 1 到 10 之间取随机数`
- **外观** `显示`
- **控制** `重复执行`
- **运动** `将 y 坐标增加`
- **控制** `如果 那么`
- **控制** `删除此克隆体`
  代码：

```
当作为克隆体启动时
移到 x: (在 -220 到 220 之间取随机数) y: 170
显示
重复执行
将 y 坐标增加 (0 - 敌人速度)
如果 <(y 坐标) < -180> 那么
删除此克隆体
结束
```

说明：

- 敌人在顶部 `y=170` 出现
- `x` 是随机的，所以每次位置不同
- `0 - 敌人速度` 表示往下走

第 7 步：测试 V3

检查这些效果：

1. 点击绿旗后，敌人会不断出现
2. 敌人从上方下来
3. 敌人位置不是固定的
4. 敌人飞出底部后消失

V3 完成标准

- 子弹功能保留
- 飞船功能保留
- 敌人不断生成并下落
  保存文件：
  `space-v3.sb3`

六、V4：实现射击命中和得分

本版本目标

子弹碰到敌人时：

- 敌人消失
- 子弹消失
- 分数加 1

第 1 步：在子弹脚本里检测碰撞

打开 `子弹` 角色，找到这段代码：

```
当作为克隆体启动时
移到 [玩家飞船]
将 y 坐标增加 20
显示
重复执行
将 y 坐标增加 12
如果 <(y 坐标) > 180> 那么
删除此克隆体
结束
```

现在在循环里面加入碰撞判断。
要用的积木：

- **侦测** `碰到 [敌人1] ?`
- **控制** `如果 那么`
- **变量** `将 [分数] 增加 1`
- **事件** `广播消息`
- **控制** `删除此克隆体`
  更新后的代码：

```
当作为克隆体启动时
移到 [玩家飞船]
将 y 坐标增加 20
显示
重复执行
将 y 坐标增加 12
如果 <碰到 [敌人1] ?> 那么
将 [分数 v] 增加 1
广播 [敌人被击中 v]
删除此克隆体
结束
如果 <(y 坐标) > 180> 那么
删除此克隆体
结束
```

第 2 步：让敌人收到“被击中”后消失

这里要注意一个 Scratch 小难点：
如果所有敌人都听到同一个广播，可能会一起乱反应。所以更简单的初学方法是：
不要靠广播来删除敌人，而是让敌人在自己的循环中检测是否碰到子弹。
所以我们把上一步稍微改成更稳的方案：
子弹脚本改为：

```
当作为克隆体启动时
移到 [玩家飞船]
将 y 坐标增加 20
显示
重复执行
将 y 坐标增加 12
如果 <(y 坐标) > 180> 那么
删除此克隆体
结束
```

然后在 `敌人1` 的克隆体循环里检测碰撞。
把敌人脚本改成：

```
当作为克隆体启动时
移到 x: (在 -220 到 220 之间取随机数) y: 170
显示
重复执行
将 y 坐标增加 (0 - 敌人速度)
如果 <碰到 [子弹] ?> 那么
将 [分数 v] 增加 1
```

```
删除此克隆体
结束
如果 <(y 坐标) < -180> 那么
删除此克隆体
结束
```

这样更适合小朋友理解：

- 敌人自己检查："我有没有碰到子弹？"
- 碰到就加分并消失

第 3 步：让子弹打中后也消失

如果只删敌人，不删子弹，可能一颗子弹连续打中多个敌人。
所以我们给 `子弹` 再加一条判断：

```
当作为克隆体启动时
移到 [玩家飞船]
将 y 坐标增加 20
显示
重复执行
将 y 坐标增加 12
如果 <碰到 [敌人1] ?> 那么
删除此克隆体
结束
如果 <(y 坐标) > 180> 那么
删除此克隆体
结束
```

第 4 步：测试 V4

看下面几点：

1. 子弹打到敌人，敌人会消失吗？
2. 子弹打到敌人，子弹会消失吗？
3. 分数会增加吗？
4. 没有打中时，敌人会继续往下飞吗？

V4 完成标准

- 游戏第一次变得"好玩"
- 已经有了"发射"和"击中"的感觉

- 分数可以正常累计
  保存文件：
  `space-v4.sb3`

七、V5：加入生命值和游戏结束

本版本目标

如果敌人撞到玩家飞船，玩家生命减 1。生命变成 0 时，游戏结束。

第 1 步：让敌人检测是否碰到玩家

在 `敌人1` 的克隆体脚本中，加入：
需要积木：

- **侦测** `碰到 [玩家飞船] ?`
- **变量** `将 [生命] 增加 -1`
- **控制** `删除此克隆体`
  代码改成：

```
当作为克隆体启动时
移到 x: (在 -220 到 220 之间取随机数) y: 170
显示
重复执行
将 y 坐标增加 (0 - 敌人速度)
如果 <碰到 [子弹] ?> 那么
将 [分数 v] 增加 1
删除此克隆体
结束
如果 <碰到 [玩家飞船] ?> 那么
将 [生命 v] 增加 -1
删除此克隆体
结束
如果 <(y 坐标) < -180> 那么
删除此克隆体
结束
```

第 2 步：让舞台判断游戏是否结束

选择 `舞台背景`，加入代码：
需要积木：

- **事件** `当绿旗被点击`
- **控制** `重复执行`

- **控制** `如果 那么`
- **运算** `=`
- **变量** `将 [游戏状态] 设为`
- **事件** `广播消息`

```
当绿旗被点击
重复执行
如果 <(生命) = 0> 那么
将 [游戏状态 v] 设为 [结束]
广播 [游戏结束 v]
结束
```

第 3 步：收到游戏结束后停止角色动作

玩家飞船

在 `玩家飞船` 的移动代码里，外面加一个判断：

```
当绿旗被点击
显示
移到 x: 0 y: -140
重复执行
如果 <(游戏状态) = [游戏中]> 那么
如果 <按下 [右箭头] 键? > 那么
将 x 坐标增加 8
结束
如果 <按下 [左箭头] 键? > 那么
将 x 坐标增加 -8
结束
如果 <(x 坐标) > 220> 那么
将 x 设为 220
结束
如果 <(x 坐标) < -220> 那么
将 x 设为 -220
结束
结束
```

子弹

在子弹发射代码中也加判断：

```
当绿旗被点击
隐藏
```

```
重复执行
如果 <<按下 [空格] 键? > 且 <(游戏状态) = [游戏中]>> 那么
克隆自己
等待 0.2 秒
结束
```

这里需要 **运算** 里的 `且`

敌人

敌人生成代码也已经用了 `游戏状态`，这样游戏结束后就不会继续生成。

第 4 步：切换到“游戏结束”背景

先准备一个新背景：

- 命名为 `游戏结束`
  在舞台加入：

```
当收到 [游戏结束 v]
切换背景到 [游戏结束 v]
```

并且在绿旗开始时先切回战场：

```
当绿旗被点击
切换背景到 [太空战场 v]
将 [分数 v] 设为 0
将 [生命 v] 设为 3
将 [游戏状态 v] 设为 [游戏中]
将 [敌人速度 v] 设为 3
```

第 5 步：测试 V5

你要故意让敌人撞你几次，检查：

1. 生命是不是会从 3 变成 2、1、0
2. 生命到 0 时，背景会不会切换
3. 游戏结束后还能不能移动和发射

V5 完成标准

- 游戏有输掉的条件
- 有了挑战感

- 分数和生命都能工作
  保存文件：
  `space-v5.sb3`

八、V6：加入开始界面和重新开始按钮

本版本目标

游戏不再一打开就直接开始，而是先显示首页，点击开始后进入战斗。

第 1 步：增加首页背景

新增背景：

- `首页`
  上面可以写：
- 《太空小战士》
- 按按钮开始游戏

第 2 步：增加开始按钮角色

新建角色：

- `开始按钮`
  可以画一个大大的绿色按钮，上面写“开始”。

第 3 步：设置游戏状态初始值

在舞台中修改绿旗脚本：

```
当绿旗被点击
切换背景到 [首页 v]
将 [分数 v] 设为 0
将 [生命 v] 设为 3
将 [游戏状态 v] 设为 [开始中]
将 [敌人速度 v] 设为 3
```

第 4 步：开始按钮的显示和点击逻辑

在 `开始按钮` 角色中：

```
当绿旗被点击
显示
移到 x: 0 y: -20
```

再加入点击代码：
需要积木：

- **事件** `当这个角色被点击`
- **变量** `将 [游戏状态] 设为`
- **外观** `隐藏`
- **外观** `切换背景到`
  因为按钮不能直接切背景，所以推荐使用广播。

```
当这个角色被点击
如果 <(游戏状态) = [开始中]> 那么
广播 [开始游戏 v]
隐藏
结束
```

第 5 步：舞台收到开始游戏广播

在舞台添加：

```
当收到 [开始游戏 v]
切换背景到 [太空战场 v]
将 [分数 v] 设为 0
将 [生命 v] 设为 3
将 [游戏状态 v] 设为 [游戏中]
```

第 6 步：游戏结束后重新开始

最简单的方法是再做一个 `重新开始按钮`，逻辑和开始按钮差不多。
如果你想先简单一点，也可以告诉小朋友：

- 先点击红色停止按钮
- 再点击绿色旗子
  但更完整的版本建议增加 `重新开始按钮`。
  按钮逻辑：

```
当收到 [游戏结束 v]
显示
移到 x: 0 y: -40
当这个角色被点击
如果 <(游戏状态) = [结束]> 那么
广播 [开始游戏 v]
```

```
隐藏
结束
```

第 7 步：测试 V6

检查流程：

1. 点击绿旗，先看到首页
2. 点击开始按钮，进入战场
3. 打到生命为 0，进入游戏结束画面
4. 可以重新开始

V6 完成标准

- 游戏体验更像真正作品
- 玩家可以开始、结束、再开始
  保存文件：
  `space-v6.sb3`

九、V7：增加关卡、难度和音效

本版本目标

分数越高，敌人越快；还能加入发射和爆炸音效。

第 1 步：创建关卡变量

点击 **变量**，新建：

- `关卡`
  在舞台初始化中加入：

```
将 [关卡 v] 设为 1
```

第 2 步：根据分数提升关卡

在舞台增加一个循环：

```
当绿旗被点击
重复执行
如果 <(分数) > 9> 那么
将 [关卡 v] 设为 2
将 [敌人速度 v] 设为 5
结束
```

```
如果 <(分数) > 19> 那么
将 [关卡 v] 设为 3
将 [敌人速度 v] 设为 7
结束
结束
```

说明：

- 得分超过 10，进入第 2 关
- 得分超过 20，进入第 3 关
- 敌人速度会变快

第 3 步：增加发射音效

选择 `子弹` 角色，切换到“声音”标签，添加一个激光音效。
然后在发射代码中加入：
需要积木：

- **声音** `播放声音 [xxx] 直到播放完毕`
- 或者 `开始播放声音 [xxx]`
  建议使用：

```
开始播放声音 [laser v]
```

放在：

```
如果 <<按下 [空格] 键? > 且 <(游戏状态) = [游戏中]>> 那么
开始播放声音 [laser v]
克隆自己
等待 0.2 秒
结束
```

第 4 步：增加敌人爆炸音效

选择 `敌人1`，添加一个爆炸声音。
在碰到子弹的代码里加入：

```
如果 <碰到 [子弹] ?> 那么
开始播放声音 [explosion v]
将 [分数 v] 增加 1
删除此克隆体
结束
```

第 5 步：增加简单爆炸效果（可选）

可以做一个 `爆炸效果` 角色：

- 有 2 到 4 个造型
- 每个造型越来越大、越来越淡
  做法：

1. 子弹击中敌人时广播 `爆炸`
2. 爆炸角色收到广播后到目标位置播放动画
   这个步骤稍微难一点，如果孩子刚学 Scratch，可以先跳过，等熟悉后再加。

第 6 步：测试 V7

检查：

1. 发射时有没有声音
2. 敌人被打中有没有声音
3. 分数高了以后，敌人是不是更快
4. 关卡变量是不是会变化

V7 完成标准

- 游戏节奏更刺激
- 难度会成长
- 玩起来更像正式游戏
  保存文件：
  `space-v7.sb3`

十、V8：加入 Boss 和胜利界面

本版本目标

当分数达到一定数量，普通敌人减少，Boss 出现。打败 Boss 后进入胜利画面。

第 1 步：创建 Boss 角色

新建角色：

- `Boss`
  建议：
- 大一些，大小 `100` 左右
- 放在舞台上方

- 造型比普通敌人更特别

第 2 步：创建 Boss 生命变量

新建变量：

- `Boss生命`
  初始化：

```
当绿旗被点击
将 [Boss生命 v] 设为 20
```

第 3 步：让 Boss 在高分时出现

在 `Boss` 角色中：

```
当绿旗被点击
隐藏
重复执行
如果 <<(分数) > 29> 且 <(游戏状态) = [游戏中]>> 那么
显示
移到 x: 0 y: 120
停止这个脚本
结束
```

说明：

- 分数大于 29，也就是至少 30 分时 Boss 出现

第 4 步：让 Boss 左右移动

在 `Boss` 中继续写：

```
当绿旗被点击
隐藏
当收到 [Boss开始 v]
显示
移到 x: 0 y: 120
重复执行
将 x 坐标增加 4
如果 <(x 坐标) > 180> 那么
面向 -90 方向
结束
如果 <(x 坐标) < -180> 那么
```

```
面向 90 方向
结束
```

不过上面的写法对三年级小朋友有一点绕，所以更推荐：

```
当收到 [Boss开始 v]
显示
移到 x: 0 y: 120
重复执行
在 1 秒内滑行到 x: 150 y: 120
在 1 秒内滑行到 x: -150 y: 120
```

这样更容易理解。

第 5 步：子弹打中 Boss 会扣 Boss 生命

在 `Boss` 角色中：

```
当绿旗被点击
重复执行
如果 <<碰到 [子弹] ?> 且 <(游戏状态) = [游戏中]>> 那么
将 [Boss生命 v] 增加 -1
结束
```

为了避免一碰到就连续掉很多血，可以加一个短暂等待：

```
当绿旗被点击
重复执行
如果 <<碰到 [子弹] ?> 且 <(游戏状态) = [游戏中]>> 那么
将 [Boss生命 v] 增加 -1
等待 0.1 秒
结束
```

第 6 步：Boss 被打败后胜利

在 `Boss` 或舞台加入：

```
当绿旗被点击
重复执行
如果 <(Boss生命) = 0> 那么
将 [游戏状态 v] 设为 [胜利]
广播 [游戏胜利 v]
结束
```

舞台收到消息后：

```
当收到 [游戏胜利 v]
切换背景到 [胜利 v]
```

第 7 步：让普通敌人在 Boss 战时减少（可选）

在 `敌人1` 的生成脚本里加判断：

```
当绿旗被点击
重复执行
如果 <<(游戏状态) = [游戏中]> 且 <(分数) < 30>> 那么
克隆自己
等待 1 秒
结束
```

这样 Boss 出现后，普通敌人不再继续刷太多，画面会更清楚。

第 8 步：测试 V8

检查：

1. 分数够高后 Boss 会不会出现
2. 子弹打 Boss，Boss 生命会不会减少
3. Boss 生命到 0 后会不会胜利
4. 胜利背景会不会切换

V8 完成标准

- 游戏有最终目标
- 有完整的开始、战斗、失败、胜利流程
  保存文件：
  `space-v8.sb3`

十一、推荐的制作顺序总结

如果你准备真正开始做，按下面顺序最稳：

1. 先做 `V1`，只做飞船移动
2. 再做 `V2`，加入发射子弹
3. 再做 `V3`，让敌人出现
4. 再做 `V4`，完成击中得分
5. 再做 `V5`，加入生命和结束

6. 再做 `V6`，加入开始和重新开始
7. 再做 `V7`，加入关卡和音效
8. 最后做 `V8`，加入 Boss 和胜利
   不要跳着做。
   原因是：

- 前面的代码是后面的基础
- 如果一下做太多，出错后不容易找原因

十二、每次做完一个版本，怎么检查有没有出错

这是最重要的好习惯。
每做完一个版本，都做这 4 件事：

1. 点击绿旗重新开始

看角色位置、变量数值是不是正常。

2. 只测一个功能

比如现在只做了移动，就只测试移动，不要同时想着子弹和敌人。

3. 出错时先想“是谁的代码有问题”

比如：

- 飞船不动，先看 `玩家飞船`
- 子弹不发射，先看 `子弹`
- 分数不加，先看 `敌人1` 或 `子弹`

4. 版本另存

每次成功后立刻保存新版本，不要覆盖所有旧版本。

十三、常见错误和解决办法

1. 子弹一直留在屏幕上

原因：

- 没有加 `删除此克隆体`
  解决：
- 在子弹飞到上面时删除它

2. 敌人不出现

原因可能有：

- 敌人角色开局隐藏了，但克隆体没有显示
- 没有写 `克隆自己`
- `游戏状态` 不是 `游戏中`
  解决：
- 检查是否在 `当作为克隆体启动时` 里用了 `显示`

3. 按空格没反应

原因：

- 发射代码没放在 `子弹` 里
- `游戏状态` 不是 `游戏中`
- 用了 `隐藏` 但没有克隆
  解决：
- 检查角色是否选对

4. 分数一直不增加

原因：

- 敌人没有检测到碰撞
- 子弹和敌人的角色名称没选对
  解决：
- 检查 `碰到 [子弹] ?` 和 `将 [分数] 增加 1`

5. 游戏结束后还在动

原因：

- 代码里没有判断 `游戏状态`
  解决：
- 在移动、发射、生成的循环里都加上状态判断

十四、适合孩子继续挑战的小扩展

如果前面的版本都完成了，还可以继续加这些功能：

- 双发子弹：一次发两颗
- 特殊敌人：更快，但分数更高
- 护盾道具：吃到后暂时无敌
- 血包道具：生命加 1

- 背景滚动：看起来像真的在飞
- 玩家有爆炸动画
- Boss 也会发射子弹
- 限时模式：60 秒内打多少分
  这些都不需要一开始做，等基础版本稳定后再加。

十五、给家长或老师的使用建议

如果这份方案是大人陪孩子一起做，建议这样安排：

- 第一次只做 `V1` 和 `V2`
- 第二次做 `V3` 和 `V4`
- 第三次做 `V5` 和 `V6`
- 第四次做 `V7` 和 `V8`
  每次控制在 `30` 到 `45` 分钟比较合适。
  孩子学 Scratch 时，最重要的不是一次做完，而是：
- 理解每块积木为什么这样接
- 学会测试
- 学会发现问题并修改

十六、最后给小朋友的话

你现在做的，不只是一个小游戏。
你做的是一个真正的“游戏工程”：

- 有角色
- 有背景
- 有变量
- 有规则
- 有版本升级
  很多真正的程序员，也是这样开发软件的：

1. 先做最小能运行版本
2. 然后一点点增加功能
3. 每次改完都测试
4. 出问题再修改
   所以你现在已经在用真正的编程方法了。
   从 `V1` 开始，慢慢做，不着急。每完成一个版本，你都会更像一名小小游戏开发者。